

Due: Sep 6, 2026 (11:59 pm)

Instructions and Policy: You are allowed to study with others and use online resources for reference, however, the work you turn in must be your own. This means do not copy/paste from Stack Exchange, ChatGPT, or another student. If you have worked closely with other students, provide their name(s) and a brief (at most one paragraph) description of the interaction; if we feel this oversteps the bounds, we will discuss it with you. Each student should write up their own solutions independently.

The requirements below are supposed to be followed in this and further homework assignments.

- For your theoretical submission:
 - **YOU MUST INCLUDE YOUR NAME IN THE HOMEWORK.**
 - The answers **MUST** be submitted via Gradescope.
 - Please write clearly and concisely. Clarity and brevity will be rewarded. Refer to known facts as necessary.
 - Theoretical questions **MUST include the intermediate steps to the final answer.**
- For your programming answers:
 - **The Python scripts will be submitted separately via Gradescope.**
 - Zero points will be given in any question where the Python code answer doesn't match the answer on Gradescope.
 - If the answer is/includes a plot, it should be added to your theoretical submission unless otherwise specified.

Your code is **REQUIRED** to run on Python 3.11 in an environment detailed in Homework 0 under the Python Installation section. While your code may run in other environments, any points lost due to environmental issues will not be regraded.

Please do not use libraries that are not imported for you. If such a library is used, you will get 0 points for the coding part of the assignment.

If your code does not run on Gradescope, then even if it compiles on another computer, the respective part of the assignment will still receive 0 points.

Python Installation

This class uses **Python 3.11**. We recommend [Miniconda](#), a small Python distribution that provides the `conda` command-line environment manager without installing the large collection of packages included with Anaconda Distribution. If `conda --version` already works on your computer, you do not need to install Miniconda again.

Otherwise, download and install the appropriate **Miniconda** installer from <https://www.anaconda.com/download>. Follow the official instructions for your operating system, allow the installer to initialize `conda` when prompted, and then close and reopen your terminal. On Windows, use the Anaconda Prompt installed with Miniconda; on macOS or Linux, use the Terminal application.

All commands below must be run in a terminal. First, verify that `conda` is available:

```
conda --version
```

Download and extract `hw0_handout.zip` before creating the course environment. In the terminal, change into the extracted `hw0_handout` directory. Replace the example path below with the actual path on your computer; place quotation marks around a path that contains spaces.

```
cd "path/to/hw0_handout"
```

Confirm that this directory contains `requirements.yml` and the three `*_basics.py` files by running `dir` on Windows or `ls` on macOS/Linux. Then create the dedicated `cs373` environment from the provided file:

```
conda env create --file requirements.yml
```

Run this creation command only once. If `conda` reports that an environment named `cs373` already exists, update that environment instead:

```
conda env update --name cs373 --file requirements.yml --prune
```

Whenever you work on this class, open a terminal and activate the environment:

```
conda activate cs373
```

Verify that the terminal is using the required interpreter. The first command must report **Python 3.11.x**, where `x` is any patch version. The second command verifies the main packages without opening an interactive Python session.

```
python --version
python -c "import numpy, pandas, matplotlib; print('packages OK')"
```

You may use VS Code, PyCharm, or another editor to change the files. If your editor asks for a Python interpreter, select the one in the `cs373` environment. Regardless of editor, run and test the scripts with the terminal commands shown in this handout; this is the closest match to how Gradescope runs your submission.

Programming Part (11+9+13=33 pts)

Overview

For this assignment, please download the provided `hw0_handout.zip` file from **Brightspace**.

In this assignment, you will learn and use some basic Python and NumPy skills. You will also learn how to load and visualize data using Pandas and Matplotlib. The goal of this assignment is to get you familiar with the tools we will use in this class.

Files

You will fill in functions in `python_basics.py`, `numpy_basics.py`, and `data_basics.py`. After you finish, you will submit these three files as well as any other files mentioned in the assignment to Gradescope.

Aside from these three files, you will also find `utils.py` and `exam_original.csv` in the same folder. You do not need to modify these two files. `utils.py` contains some helper functions, and `exam_original.csv` is the data you will use in this assignment.

Packages

You will need the following packages for this assignment:

- `numpy`
- `pandas`
- `matplotlib`

These packages are installed into the `cs373` environment by the provided `requirements.yml` file. Do not install them one at a time or into your base environment.

Evaluation

Unless otherwise specified, your code will be evaluated by an autograder on Gradescope using the same environment as detailed above. You should make sure your code runs without errors in that environment. Otherwise, you may lose points. You can submit your code to Gradescope as many times as you want. We will only grade the latest submission.

There will be two types of test cases in the autograder: public and hidden. Public test results are visible after you submit your code. The provided scripts also contain local doctests that you can run from the terminal with `python <filename>.py` while your terminal is in the `hw0_handout` directory. Passing all local doctests and public tests **does not** guarantee you will get full credit for the assignment.

Hidden test cases, however, are not visible to you, and you will not be able to see the results until your grade is published. The final score you get for this assignment is the sum of the scores of all public and hidden test cases.

Getting Help

If you have any questions about this assignment, please contact the course staff for help, preferably during office hours. You can also post your questions on the course forum. However, please do not post any code publicly. When asking questions, you should describe your problem in words and post the relevant code snippet. Please avoid showing a screenshot of your code to TAs and ask “why my code is not working”. This will not help you get an answer faster.

1 Python Basics (11 pts)

Getting Started

Keep all provided files together in the extracted `hw0_handout` directory. In a terminal, activate the environment and change into that directory before running any script:

```
conda activate cs373
cd "path/to/hw0_handout"
python python_basics.py
```

Use your editor to change `python_basics.py`, then return to the terminal and rerun the last command. A correct solution prints `All tests passed!`. Follow the same edit-run cycle for the other two scripts later in the assignment:

```
python numpy_basics.py
python data_basics.py
```

Do not use Jupyter Notebook or a similar interactive tool for this assignment. Gradescope executes complete `.py` files from a terminal-like environment, so variables or imports created only in an interactive session will not be available.

Understanding Python Type Hints and DocTests

While you may have some Python background, you may not have seen the following syntax before. This is called [type hints](#) and is used to specify the types of the input and output of a function. Type hints are not required nor enforced by Python but are meant to help you understand the code better. For example, the function signature below means that the function `mystery_function` takes two integers `a` and `b` and a tuple of two integers as input and returns an integer.

```
1 def mystery_function(a: int, b: int, c: Tuple[int, int]) -> int:
2     """
3     This is a docstring. It is used to describe the function.
4     Below are some examples of doctests.
5     >>> mystery_function(1, 2, (3, 4))
6     10
7     >>> mystery_function(1, 2, (3, 4, 5)) # This works but does not follow the type hints.
8     10
9     >>> mystery_function(1, 2) # This would cause an error.
10    Traceback (most recent call last):
11    ...
12    TypeError: mystery_function() missing 1 required positional argument: 'c'
13    """
14    return a + b + c[0] + c[1]
```

You should also notice that we have added some tests to the docstrings of the functions. These tests are called `doctests`. The code following `>>>` is the input to a Python interpreter, and the code after that is the expected output. The doctests in the skeleton code serve as examples and local tests. Run them from the `hw0_handout` directory with `python python_basics.py`.

Doctests compare expected and actual output exactly. If a local test fails, read the displayed difference, correct your code, and rerun the script. Gradescope also runs additional tests, so a passing local result is necessary but not sufficient for full credit.

1.1 (2 pts) OOP in Python - Constructor

In class `Exponentiation`, fill in the constructor (`__init__`) so that it takes `base` and the integer `power` as input and stores them as attributes `self.base` and `self.power`. You do not need to return anything. Notice the keyword `self`, which is a reference to the current instance of the class and is similar to `this` in Java.

1.2 (1 pt) OOP in Python - Method

In class `Exponentiation`, fill in the method `compute` so that it calculates `self.base` raised to `self.power`, stores the value as `self.result`, and returns that value.

1.3 (2 pts) Print and Format

In class `Exponentiation`, fill in the method `display` so that it prints the answer in the following format:

```
Base: 2.00
Power: 3
Result: 8.0000
```

Note that the base should be printed with two decimal places, the power should be printed as integer, and the value for the result should be printed with four decimal places. There are many ways to do this. You can use `print` and `format` or `f-string`. You can also use `round` to round the answer to four decimal places.

1.4 (2 pts) Data Structures - List

In class `DataStructure`, fill in the method `generate_list` so that it returns a list containing 8 random integers between -5 and 5 (inclusive). You should use `random.randint` to generate random integers. Do not include `random.seed()` in your code!

1.5 (2 pts) Data Structures - Set

In class `DataStructure`, fill in the method `find_non_negative` so that it returns a set containing all values in `my_list` that are non-negative.

1.6 (2 pts) Data Structures - Dictionary

In class `DataStructure`, fill in the method `count_signs` so that it returns a dictionary containing the count of negative, zero, and positive values in `my_list`. The dictionary must have exactly the keys "neg", "zero", and "pos", as shown in the function's doctests.

2 NumPy Basics (9 pts)

In this part, you will look through the NumPy [Quickstart Tutorial](#) and learn how to use NumPy. You will fill in the functions in `numpy_basics.py`. From the `hw0_handout` directory, run the local doctests with `python numpy_basics.py`.

2.1 (3 pts) NumPy Basics - Array

In class `NumpyBasics`, fill in the method `split_even_odd_values` so that it splits the input list or array `x` into two arrays `even_values` and `odd_values` and returns them. The array `even_values` contains all the even values in input `x`. The array `odd_values` contains all the odd values in input `x`. **DO NOT use any loops.**

Note that you should use NumPy arrays instead of Python lists. You can use `np.array` to create a NumPy array from a Python list when needed.

2.2 (3 pts) NumPy Basics - Functions

In class `NumpyBasics`, fill in the method `regularized_transpose_product_inverse` so that it takes a NumPy array `x` and a scalar float value `lam` as input and returns the inverse of the matrix $(\mathbf{x} \times \mathbf{x}^T + \lambda \mathbf{I})$. Here the superscript $\top$ denotes the transpose of the input array `x` and `I` denotes an identity matrix of the appropriate size. Feel free to use `np.linalg.inv` to compute the inverse. **DO NOT use any loops.**

2.3 (3 pts) NumPy Basics - Broadcasting

In class `NumpyBasics`, fill in the method `nonnegative_max_normalization` that applies the following normalization to the input NumPy array `x` and returns the normalized array:

- Replace all negative values with 0.
- Divide the updated array by its maximum value and then return the normalized array.
- If the maximum value is 0, return an array of zeros.

DO NOT use any loops.

3 Playing around with Data (13 pts)

In this part, you will learn how to load and visualize data using [Pandas](#) and [Matplotlib](#). You will fill in the functions in `data_basics.py`. From the `hw0_handout` directory, run the local doctests with `python data_basics.py`.

3.1 (1 pt) Loading Data

In class `DataBasics`, fill in the constructor (`__init__`) so that it loads the data from the CSV file named `filename` into a Pandas DataFrame and stores it as an attribute `self.df`. You can use `pd.read_csv` to load the data.

3.2 (3 pts) Preprocessing Data

In class `DataBasics`, fill in the method `preprocess` so that it updates the DataFrame stored in `self.df` in place. The method returns nothing. Since this is an introductory assignment, the preprocessing steps are simple:

- Drop the column `Unnamed: 0`.
- Rename the column `ParentEduc` to `ParentEducation`

After preprocessing, print the first 3 rows of the DataFrame.

3.3 (2 pts) Exploring Data

In class `DataBasics`, fill in the method `writing_stats_given_score` so that it returns the minimum, maximum, and median of the math score for students who scored less than or equal to `score` in writing.

3.4 (3 pts) Visualizing Data - Histogram

In class `DataBasics`, fill in the method `generate_reading_score_histogram` so that it plots a histogram of the reading score. The histogram should have 15 bins and include appropriate labels and title. The graph is automatically saved as `reading_score_histogram.png` in the same folder as your code after you run `python data_basics.py`. Don't forget to upload the graph to Gradescope, where the TAs will grade your graph manually.

3.5 (4 pts) Visualizing Data - Scatter Plot

In class `DataBasics`, fill in the method `generate_math_writing_scatterplot` so that it plots a scatter plot of the math score and writing score (writing score in y-axis vs math score in x-axis). The scatter plot should have appropriate labels and a title. You also need to set the edge color of the points to white and transparency to 0.95 so that the points are easier to see. After that, add a regression line in the scatter plot. You may find `np.polyfit` useful.

The graph is automatically saved as `math_writing_scatterplot.png` in the same folder as your code after you run `python data_basics.py`. Don't forget to upload the graph to Gradescope, where the TAs will grade your graph manually.

Submission

You will submit the following files to Gradescope. Do not put your source files in a nested folder.

- `python_basics.py`
- `numpy_basics.py`
- `data_basics.py`

You will also submit the following graphs to the separate HW 0 - Plots assignment on Gradescope:

- `reading_score_histogram.png`
- `math_writing_scatterplot.png`